

Bases de Datos a Gran Escala

Sesión 3 · Introducción a los sistemas NoSQL

Diego Sevilla Ruiz

Departamento de Ingeniería y Tecnología de Computadores

Facultad de Informática · Universidad de Murcia

dsevilla@um.es

Introducción a los sistemas NoSQL

Introducción a NoSQL

- **NoSQL** → *hashtag* llamativo elegido para una conferencia en 2009 (Johan Oskarsson, de Last.fm)
- Ahora se asocia a cientos de bases de datos diferentes, clasificadas en varios tipos (los vemos después), caracterizadas por **no usar SQL** como modelo de datos
- Más recientemente, **NoSQL** → **Not Only SQL** (no sólo SQL) → persistencia políglota (*polyglot persistence*)

NoSQL





Your Ultimate Guide to the
Non-Relational Universe!

[including a historic [Archive](#) 2009-2011]
News Feed covering some changes [here](#) !

NoSQL DEFINITION:Next Generation Databases mostly addressing some of the points: being non-relational, distributed, open-source and horizontally scalable.

The original intention has been **modern web-scale databases**. The movement began early 2009 and is growing rapidly. Often more characteristics apply such as: **schema-free, easy replication support, simple API, eventually consistent / BASE (not ACID), a huge amount of data** and more. So the misleading term "*nosql*" (the community now translates it mostly with "**not only sql**") should be seen as an alias to something like the definition above. [based on 7 sources, 15 constructive feedback emails (thanks!) and 1 disliking comment . Agree / Disagree? [Tell](#) me so! By the way: this is a strong definition and it is out there here since 2009!]

LIST OF NOSQL DATABASES [currently >225]

Core NoSQL Systems: [Mostly originated out of a Web 2.0 need]

Wide Column Store / Column Families

Hadoop / HBase API: Java / any writer, Protocol: any write call, Query Method: MapReduce Java / any exec, Replication: HDFS Replication, Written in: Java, Concurrency: ?, Misc: Links: 3 Books [\[1, 2, 3\]](#)

MapR, Hortonworks, Cloudera Hadoop Distribution and professional services .

Cassandra massively scalable, partitioned row store, masterless architecture, linear scale performance, no single points of failure, read/write support across multiple data centers & cloud availability zones. API / Query Method: CQL and Thrift, replication: **peer-to-peer**, written in: Java, Concurrency: **tunable consistency**, Misc: built-in data compression, MapReduce support, primary/secondary indexes, security features. Links: [Documentation](#), [PlanetC*](#), [Company](#).

Scylla Cassandra-compatible column store, with consistent low latency and more transactions per second. Designed with a thread-per-core model to maximize performance on modern multicore

NoSQL RELATED EVENTS:

- 12/14 Sept Flink Forward »
- 26 Sept Big Data Guide Event Frankfurt »



Register your event 4free: »

NoSQL ARCHIVE



NoSQL FORUMS

- Global NOSQL Forum »
- Forum Berlin »
- Forum France »
- Forum Japan »

nosql-databases.org

NoSQL: ¿por qué se plantearon?

1. Mayor escalabilidad horizontal

- Conjuntos de datos muy muy grandes
- Sistemas de alto volumen de escrituras (*streaming* de eventos, aplicaciones sociales)

2. Demanda de productos de software libre (crecimiento de las *start-ups*)

3. Consultas especializadas no eficientes en el modelo relacional (JOIN s)

4. Expresividad, flexibilidad, dinamismo: frustración con las restricciones del modelo relacional



NoSQL: características

- No se basan en SQL
- Modelos de datos más ricos
- Orientadas a la **escalabilidad**
- Generalmente no obligan a definir un esquema → ***schemaless***
- Surgidas de la comunidad para solucionar problemas → muchas ***libres/open source***
- Diseño basado en **procesamiento distribuido**
- Principios funcionales → **MapReduce**

Categorías de NoSQL

- Bases de datos *key-value*
- Bases de datos documentales
- Bases de datos columnares (*wide column*)
- Bases de datos de grafos
- Bases de datos de *arrays*

Evolución desde el modelo relacional

- El **modelo relacional** $\Rightarrow$ **predominante en los últimos ~30 años**
- Tiene sus raíces en el denominado *business data processing*, procesamiento de transacciones y *batch*
- Propuesto por Codd en los 70, **de alto nivel**
- Actualmente los **sistemas SQL están muy optimizados**:
 - El **grado de implantación es mayoritario**
 - Para el 99 % de los problemas (que caben en un ordenador) es eficiente y adecuado

Adopción de NoSQL

Twitter cambiando a Cassandra por 2010. Cassandra, desarrollada en Facebook en 2009

The screenshot shows a Computerworld article page. At the top, there's a navigation bar with the Computerworld logo and a 'TRENDING' section. The article title is 'Twitter growth prompts switch from MySQL to 'NoSQL' database'. Below the title is a subtitle: 'Hopes open source technology can further boost improving Twitter uptime record'. The author is 'By Eric Lai' and the date is 'Feb 23, 2010 4:10 PM PT'. To the right of the article, there's a 'MORE LIKE THIS' section with three related articles. At the bottom left, there's a 'RELATED TOPICS' section with two topics: 'Database Management' and 'Enterprise Applications'.

TRENDING: What you need to know about Windows 10's new upgrade · First impressions: iPad Pro 'wicked fast' · Resources/White Papers

COMPUTERWORLD

Popular Now: [dropdown]

Home > Enterprise Applications > Database Management

NEWS

Twitter growth prompts switch from MySQL to 'NoSQL' database

Hopes open source technology can further boost improving Twitter uptime record

By Eric Lai

Computerworld | Feb 23, 2010 4:10 PM PT

RELATED TOPICS

- Database Management
- Enterprise Applications

MORE LIKE THIS

- The end of SQL and relational databases? (part 3 of 3)
- InfoWorld's top 10 emerging enterprise technologies
- InfoWorld review: Databases primed for social networks

RELATED TOPICS

- Database Management
- Enterprise Applications

RELATED ARTICLES

- [Ryan King, an engineer at Twitter](#), today told the blog MyNoSQL that the social networking company plans to move from MySQL to the [Cassandra database](#) for what he called its resilience, scalability and large community of open-source developers.

Adopción NoSQL (septiembre 2020)

<https://db-engines.com/en/ranking/>

358 systems in ranking, September 2020

Rank			DBMS	Database Model	Score		
Sep 2020	Aug 2020	Sep 2019			Sep 2020	Aug 2020	Sep 2019
1.	1.	1.	Oracle +	Relational, Multi-model ⓘ	1369.36	+14.21	+22.71
2.	2.	2.	MySQL +	Relational, Multi-model ⓘ	1264.25	+2.67	-14.83
3.	3.	3.	Microsoft SQL Server +	Relational, Multi-model ⓘ	1062.76	-13.12	-22.30
4.	4.	4.	PostgreSQL +	Relational, Multi-model ⓘ	542.29	+5.52	+60.04
5.	5.	5.	MongoDB +	Document, Multi-model ⓘ	446.48	+2.92	+36.42
6.	6.	6.	IBM Db2 +	Relational, Multi-model ⓘ	161.24	-1.21	-10.32
7.	7.	↑ 8.	Redis +	Key-value, Multi-model ⓘ	151.86	-1.02	+9.95
8.	8.	↓ 7.	Elasticsearch +	Search engine, Multi-model ⓘ	150.50	-1.82	+1.23
9.	9.	↑ 11.	SQLite +	Relational	126.68	-0.14	+3.31
10.	↑ 11.	10.	Cassandra +	Wide column	119.18	-0.66	-4.22
11.	↓ 10.	↓ 9.	Microsoft Access	Relational	118.45	-1.41	-14.26
12.	12.	↑ 13.	MariaDB +	Relational, Multi-model ⓘ	91.61	+0.69	+5.54
13.	13.	↓ 12.	Splunk	Search engine	87.90	-2.01	+0.89
14.	14.	↑ 15.	Teradata +	Relational, Multi-model ⓘ	76.39	-0.39	-0.57
15.	15.	↓ 14.	Hive	Relational	71.17	-4.12	-11.93
16.	16.	↑ 18.	Amazon DynamoDB +	Multi-model ⓘ	66.18	+1.43	+8.36
17.	17.	↑ 25.	Microsoft Azure SQL Database	Relational, Multi-model ⓘ	60.45	+3.60	+32.91
18.	18.	↑ 19.	SAP Adaptive Server	Relational	54.01	+0.05	-2.09
19.	19.	↑ 21.	SAP HANA +	Relational, Multi-model ⓘ	52.86	-0.26	-2.53
20.	20.	↓ 16.	Solr	Search engine	51.62	-0.08	-7.35
21.	21.	↑ 22.	Neo4j +	Graph	50.63	+0.44	+2.41
22.	22.	↓ 20.	HBase +	Wide column	48.35	-0.76	-7.37

Tendencia septiembre 2020

<https://db-engines.com/en/ranking/>

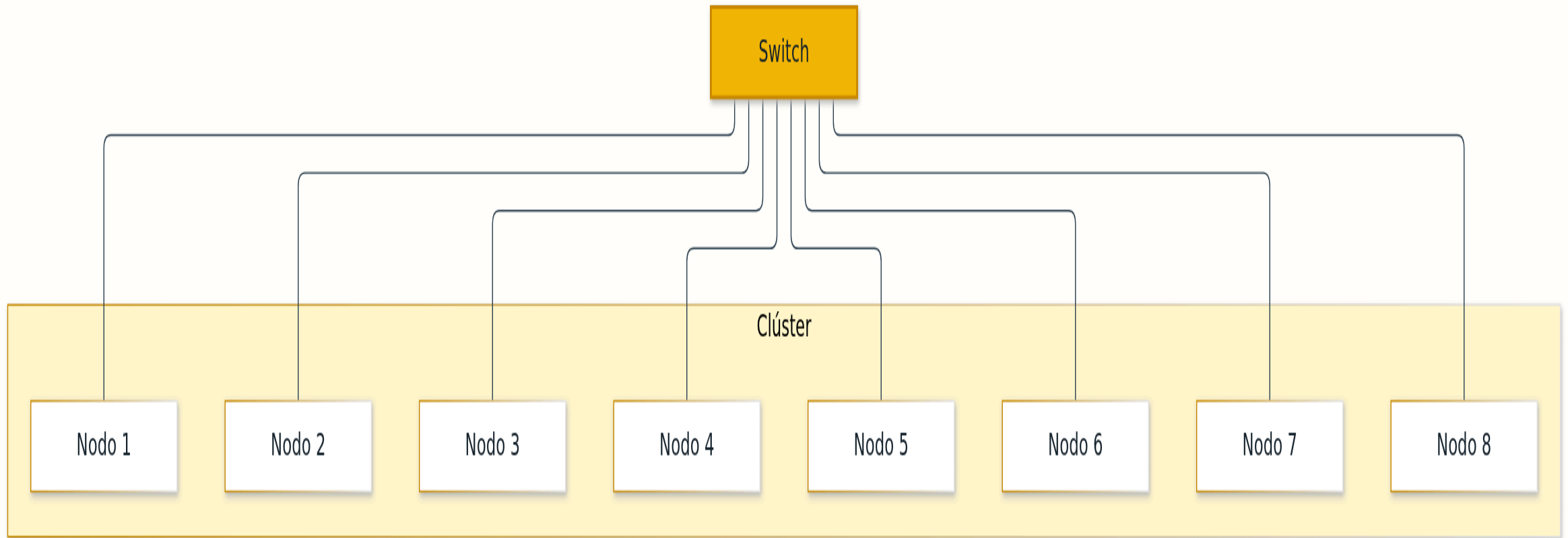


Adopción NoSQL: análisis

- Dominan los grandes SGBDR
- El *open source* tiene una importancia crucial (PostgreSQL, MySQL, MongoDB, etc.)
- Varias bases de datos NoSQL entre las 10 primeras; muchas en las 20 primeras
- La distancia entre los grandes SGBDR y el primer NoSQL (MongoDB) es de 5x
- Paradigmas más «atrevidos», como el de grafos, están entre los 20 primeros (Neo4j)

Cambio de perspectiva: red

Un clúster de nodos conectados por un *switch*: cada ordenador deja de ser sólo cómputo y pasa también a ser almacenamiento



Almacenamiento distribuido

Retomando el clúster de la transparencia anterior:

- Desde los años 90: clústers/NOC/COW — procesamiento masivamente paralelo
- **Sin embargo...** ese procesamiento asumía almacenamiento *no* distribuido
- Ahora los nodos $\Rightarrow$ también **almacenamiento**
- Minimizar el verdadero cuello de botella: **el trasiego de información por la red**

Procesamiento distribuido

- Necesidad de **paralelización máxima**
- **Escalabilidad**
- Explotar la **localidad de los datos**:
 - Los datos producidos se utilizan localmente en las siguientes iteraciones
 - Los datos recibidos llegan directamente a los *hosts* (clientes simultáneos)

Procesamiento distribuido (ii)

- Vuelta al modelo funcional inherentemente paralelo (p. ej. **Map-Reduce**)
- Almacenamiento distribuido (p. ej. **HDFS**)
- Coordinación distribuida (p. ej. **Zookeeper**)

Modelo de datos

- El modelo relacional limita a tablas con valores primitivos y relaciones *primary key/foreign key*
- En programación se utilizan **listas, arrays, tipos de datos compuestos** (*gap* semántico)
- ACID es **muy compleja y costosa** en entornos distribuidos (quizá **no necesaria** en algunas aplicaciones)

Modelo de datos (ii)

¿Y si se pudiera ver como un **gran array**?

- Cada nodo almacenaría una parte del array
- Búsqueda aleatoria **muy rápida** (árboles B)
- Uso de **filtros de Bloom**
- Uso de **objetos complejos** (p. ej. **documentos JSON**), para mantener la **localidad espacial de datos relacionados** (más adelante)
- Transacciones limitadas al **objeto complejo**

Schemaless

- Las BBDD NoSQL (en general) **no requieren de un esquema**
- **Flexibilidad:** posibilidad de almacenar entidades con una estructura diferente
 - Tratar información incompleta
 - Evolucionar la base de datos/esquema
 - Añadir nuevas características a las aplicaciones

Schema-on-write frente a schema-on-read

	(SQL)	(NoSQL)
Cuándo	Los datos conforman al esquema al escribir	Los datos leídos conforman a un esquema implícito
Tipado	Estricto (estático)	<i>Duck-typing</i> (dinámico)
Datos	Homogéneos	Heterogéneos
Proceso	Analítico, a través de consultas	<i>Use as read</i>

Schemaless: ejemplo

Añadir el campo `first_name` a partir del campo `name`. Los nuevos objetos se crean con el nuevo formato; a la hora de leerlos:

```
if (user && user.name && !user.first_name) {  
    // Docs anteriores a 2013 no tienen first_name  
    user.first_name = user.name.split(" ")[0];  
}
```

Schemaless: el mismo cambio en SQL

Puede ser un proceso muy costoso (procesa toda la tabla, *locking*, puede que haya que parar las aplicaciones):

```
ALTER TABLE users ADD COLUMN first_name text;  
UPDATE users SET first_name =  
    substring_index(name, ' ', 1);
```

¿Cuándo es apropiado *schemaless*?

- **Objetos heterogéneos**
- Estructura de los datos **impuesta externamente**
- Si intuimos que los datos **cambiarán en el futuro**

Sin embargo... a veces un esquema es conveniente

Facilita el desarrollo y evita inconsistencias. `Mongoose` , para MongoDB:

```
var Comment = new Schema({
  name: { type: String, default: 'Anonymous' },
  date: { type: Date, default: Date.now },
  text: Buffer
});
// a setter with on-line modification
Comment.path('name').set(function (v) {
  return capitalize(v);
});
```

Choose wisely



- SQL
- NoSQL
- *Polyglot persistence*

Map-Reduce

Origen en **lenguajes funcionales**:

- `map( )` : ejecuta una misma función sobre todos los elementos de un conjunto
- `reduce( )` : sumariza un conjunto de valores para producir un valor de salida

Map-Reduce: combinando ambas operaciones

- Una misma operación `map( )` sobre cada dato residente en un nodo se realiza en paralelo en **todos** los nodos simultáneamente
- Con los resultados parciales, una función `reduce( )` genera el resultado final
- Proceso intermedio de *shuffle*, para agrupar valores **con la misma clave** antes del `reduce( )`
- Resultados parciales en el mismo nodo (localidad) $\Rightarrow$ procesamientos **en cadena**

Map-Reduce



Map-Reduce — *Word Count*

<http://www.milanor.net/blog/an-example-of-mapreduce-with-rmr2/>



Map-Reduce: particularidades

- Se supone que los datos de entrada son siempre pares $\langle key, value \rangle$
- La función `map()` produce otro conjunto de valores $\{\langle key1, value1 \rangle, \langle key2, value2 \rangle, \dots\}$
- El *shuffle* agrupa los valores con la misma clave: $\{\langle key1, \{val1, val3, \dots\} \rangle, \langle key2, \{val2, val4, \dots\} \rangle, \dots\}$
- `reduce()` procesa cada lista de valores con la misma clave, y produce otros elementos $\langle key', value' \rangle$
- Hay procesamientos difíciles de expresar en Map-Reduce $\Rightarrow$ operaciones M/R **en cadena**
- Se verá en profundidad en los siguientes temas de la asignatura

Map-Reduce y consultas

Map-Reduce puede usarse no sólo para computación distribuida, sino también como una **generalización de consultas**. Ejemplo: un biólogo marino anota cada animal que ve en el océano y quiere saber cuántos tiburones ha visto por mes:

```
SELECT MONTH(observation_timestamp) AS observation_month,  
       sum(num_animals) AS total_animals  
FROM observations  
WHERE family = 'Sharks'  
GROUP BY observation_month;
```

Map-Reduce y consultas (ii)

MongoDB, con el API de MapReduce:

```
db.observations.mapReduce(  
  function map() {  
    var year = this.observationTimestamp.getFullYear();  
    var month = this.observationTimestamp.getMonth() + 1;  
    emit(year + "-" + month, this.numAnimals);  
  },  
  function reduce(key, values) {  
    return Array.sum(values);  
  },  
  {  
    query: { family: "Sharks" },  
    out: "monthlySharkReport"  
  }  
);
```

Map-Reduce y consultas (iii)

MongoDB ofrece además un API alternativo para funciones de agregación:

```
db.observations.aggregate([
  { $match: { family: "Sharks" } },
  { $group: {
    _id: {
      year: { $year: "$observationTimestamp" },
      month: { $month: "$observationTimestamp" }
    },
    totalAnimals: { $sum: "$numAnimals" }
  }
}]
```

mySQL

```
SELECT
  Dim1, Dim2,
  SUM(Measure1) AS MSum,
  COUNT(*) AS RecordCount,
  AVG(Measure2) AS MAvg,
  MIN(Measure1) AS MMin,
  MAX(CASE
    WHEN Measure2 < 100
    THEN Measure2
  END) AS MMax
FROM DenormAggTable
WHERE (Filter1 IN ('A','B'))
AND (Filter2 = 'C')
AND (Filter3 > 123)
GROUP BY Dim1, Dim2
HAVING (MMin > 0)
ORDER BY RecordCount DESC
LIMIT 4, 8
```

- ① Grouped dimension columns are pulled out as keys in the map function, reducing the size of the working set.
- ② Measures must be manually aggregated.
- ③ Aggregates depending on record counts must wait until finalization.
- ④ Measures can use procedural logic.
- ⑤ Filters have an ORM/ActiveRecord-looking style.
- ⑥ Aggregate filtering must be applied to the result set, not in the map/reduce.
- ⑦ Ascending; Descending: -1

MongoDB

```
db.runCommand({
  mapreduce: "DenormAggCollection",
  query: {
    filter1: { '$in': [ 'A', 'B' ] },
    filter2: 'C',
    filter3: { '$gt': 123 }
  },
  map: function() { emit(
    { d1: this.Dim1, d2: this.Dim2 },
    { msum: this.measure1, recs: 1, mmin: this.measure1,
      mmax: this.measure2 < 100 ? this.measure2 : 0 }
  ); },
  reduce: function(key, vals) {
    var ret = { msum: 0, recs: 0, mmin: 0, mmax: 0 };
    for(var i = 0; i < vals.length; i++) {
      ret.msum += vals[i].msum;
      ret.recs += vals[i].recs;
      if(vals[i].mmin < ret.mmin) ret.mmin = vals[i].mmin;
      if((vals[i].mmax < 100) && (vals[i].mmax > ret.mmax))
        ret.mmax = vals[i].mmax;
    }
    return ret;
  },
  finalize: function(key, val) {
    val.mavg = val.msum / val.recs;
    return val;
  },
  out: 'result1',
  verbose: true
});
db.result1.
  find({ mmin: { '$gt': 0 } }).
  sort({ recs: -1 }).
  skip(4).
  limit(8);
```

Revision 4, Created 2010-03-06
Rick Osborne, rickosborne.org

Eficiencia *raw*

- Los sistemas NoSQL también tienen que competir con los SQL en términos de eficiencia neta (o *raw*)
- Un pequeño test sintético puede ayudarnos a hacernos una idea (fuente: bonesmoses.org)
- Estos resultados están también en la hoja del repositorio de la asignatura (`addendum/mysql_vs_mongo/perf_plot.ipynb`)

Eficiencia *raw*: la prueba

- La prueba se realizó sobre MongoDB y sobre MySQL (adaptada del original, que era para PostgreSQL)
- Se parte de una tabla sencilla con cuatro valores, que muestran medidas de sensores con localización, valor de la lectura y una marca de tiempo
- Se realizan seis pruebas que pueden corresponder a un conjunto de consultas normales

Eficiencia *raw*: las seis pruebas

1. Insertar un millón de elementos generados al azar, con fechas que permitan la búsqueda por rango (**Fill**)
2. Crear un índice en la tabla para la fecha de la lectura (**Index**)
3. Actualizar los valores de un conjunto de entradas seleccionadas por rango de fechas (**Update**)
4. Eliminar un conjunto de filas seleccionadas por rango de fechas (**Delete**)
5. Obtener el número de filas restantes (**Count**)
6. Obtener un **subconjunto** de filas dado por un rango de fechas (**Interval**)



Eficiencia *raw*: análisis

- El gráfico se muestra en escala logarítmica en el eje Y (las diferencias pequeñas se acentúan)
- A simple vista, ambos productos están muy igualados:
 - SQL (MySQL) lleva *muchos años* de optimizaciones
 - Productos como MongoDB tienen menos historia de optimizaciones a sus espaldas...

Eficiencia *raw*: análisis (ii)

- Hay casos en los que uno es más rápido que el otro, y viceversa
- No se puede decir cuál es mejor:
 - **⇒ Depende del patrón de accesos que vaya a tener nuestra aplicación**
 - (p. ej., contar en MongoDB es mucho más rápido que en MySQL; la actualización es algo más rápida)

Tipos de sistemas NoSQL

Tipos de sistemas NoSQL

- **NoSQL** incluye un conjunto de tecnologías relativamente dispares
- Aún así, la mayoría comparten una serie de características:
 - No se basan en SQL
 - Generalmente no obligan a definir un esquema
 - Surgen de la comunidad para solucionar problemas, y muchas son libres/*open source*
 - Diseño basado en procesamiento distribuido, y aplican tecnologías funcionales como MapReduce
- Divididas en subcategorías: *key-value* y documentales, columnares, de grafos y de *arrays*

Key-Value Stores y documentales

- Cada pieza de datos tiene asignado un identificador
- La diferencia entre ambas es que:
 - En *key-value*, el valor es opaco: no se conoce nada de su interior (a todos los efectos es un *blob* de datos)
 - En las basadas en documentos, la base de datos puede ver el contenido del agregado, y utilizar su información como parte de las búsquedas y actualizaciones
- Documentos $\Rightarrow$ formatos jerárquicos tipo JSON o XML

Key-Value Stores y documentales (ii)

- La diferencia entre ambas queda un poco difusa:
 - Riak es *key-value*, pero permite búsquedas indexadas parecidas a las de Solr/Lucene
 - Redis permite que los valores de datos sean estructurados en *arrays*, estructuras complejas, mapas
- *Key-Value*: Riak, Redis, Memcache, LevelDB
- Documentos: Couchbase, MongoDB, OrientDB

Bases de datos columnares

- Influenciadas por el *paper* de Google de 2004 sobre BigTable
- En general, parecidas a las tablas SQL, salvo que cada fila puede:
 - Tener un conjunto de columnas diferente
 - Almacenar *series temporales* dentro de una misma fila (varias *versiones* de un mismo conjunto de columnas)
- Cada fila tiene un identificador y es un agregado de familias de columnas (*column family*)

Bases de datos columnares (ii)

- Cambian el modo de almacenamiento para favorecer ciertas aplicaciones (almacenamiento por columnas en vez de por filas)
- Bases de datos: HBase, Cassandra, Vertica, H-Store



Bases de datos de grafos

- Las bases de datos de grafos llevan el mecanismo *muchos a muchos* al extremo
- Datos en los que existen muchas relaciones entre sí, con un significado primordial
- Se basan en la construcción y consulta de grafos que constan de:
 - **Vértices**, también llamados *nodos* o *entidades*
 - **Aristas** (*edges*), también llamadas *relaciones*

Bases de datos de grafos (ii)

- Pueden capturar relaciones complejas entre entidades y ofrecen lenguajes de búsqueda, actualización y creación que permiten trabajar con subconjuntos del grafo
- Origen en las bases de datos de hechos (*Datalog*)
- Ejemplos: FlockDB, Neo4j, OrientDB



Grafos representados en SQL

Nota: usa la sintaxis PostgreSQL para `json`

```
CREATE TABLE vertices (  
    vertex_id integer PRIMARY KEY,  
    properties json  
);  
  
CREATE TABLE edges (  
    edge_id integer PRIMARY KEY,  
    tail_vertex integer REFERENCES vertices (vertex_id),  
    head_vertex integer REFERENCES vertices (vertex_id),  
    label text,  
    properties json  
);  
  
CREATE INDEX edges_tails ON edges (tail_vertex);  
CREATE INDEX edges_heads ON edges (head_vertex);
```

Grafo y consulta en Neo4j

```
CREATE
  (NAmerica:Location {name:'North America', type:'continent'}),
  (USA:Location {name:'United States', type:'country' }),
  (Idaho:Location {name:'Idaho', type:'state' }),
  (Lucy:Person {name:'Lucy' }),
  (Idaho)-[:WITHIN]->(USA)-[:WITHIN]-> (NAmerica),
  (Lucy) -[:BORN_IN]-> (Idaho)
```

Consulta:

```
MATCH
  (person) -[:BORN_IN]-> () -[:WITHIN*0..]-> (us:Location {name:'United States'}),
  (person) -[:LIVES_IN]-> () -[:WITHIN*0..]-> (eu:Location {name:'Europe'})
RETURN person.name
```

Bases de datos basadas en *arrays*

- Suelen presentarse como bases de datos que soportan SQL y añaden operaciones para trabajar con conjuntos de datos especiales (*arrays*)
- Utilizadas para tratamiento de grandes cantidades de datos de forma estadística, de modelado y OLAP
- Soportan también datos geográficos, ya que pueden definir rangos numéricos de una o varias dimensiones (2D para cálculos geográficos)
- Ejemplos: MonetDB, SciDB, rasdaman